

گزارش تمرین عملی ۴

فایل `hw4_sgd` مربوط به سوال اول، `hw4_ridge` مربوط به سوال دوم و چهارم و فایل `hw4_norm1` مربوط به سوال ۳ است.

۵۰ خط اول همه فایل‌ها مشترک است و مربوط به تمیز کردن داده‌ها و پلات کردن نتایج است.

بخش مشترک:

تابع `mse`: دو تالیست `y` واقعی و `y` پیش‌بینی شده را می‌گیرد و `MSE` را حساب می‌کند.
تابع `split_xy`: داده‌ها را می‌گیرد و `x`ها و `y`ها را جدا می‌کند و بر می‌گرداند.

در خط‌های ۲۲ تا ۴۰ (البته بین سه فایل شاید در حد یک خط به خاطر `import` های اول تفاوت باشد) اول فایل خوانده می‌شود، بعد ستون‌هایی که لازم نیستند کنار گذاشته می‌شوند. بعد متغیرهای کیفی به ستون‌های `one hot` تبدیل می‌شوند. بعد داده‌ها نرمالایز می‌شوند و `na`ها کنار گذاشته می‌شوند (البته اصلاً `na` نداشت) بعد هم به سه دسته `train`, `val`, `test` تقسیم می‌شوند و `x` و `y` هایشان جدا می‌شود.

تابع `Plot`: لیست مقادیر `mse` در داده‌های `validation` و تست به ازای دورهای مختلف آپدیت شدن بردار وزن‌ها را می‌گیرد و نمودار خطی آنها را برحسب زمان (شماره دور) می‌کشد.

دو فایل `hw4_norm1` و `hw4_sgd` مثل هم هستند فقط در بخش آپدیت کردن بردار وزن‌ها مقدار مشتقی که کم می‌شود یک تفاوت جزئی دارد، آن هم این است که به ازای هر نمونه، مشتق خطای آن نمونه بر حسب بردار وزن‌ها تقسیم بر دو برابر قدرمطلق خطای آن نمونه می‌شود.

این دو فایل سه تابع دارند:

`Update coef`: بردار وزن‌ها را می‌گیرد و `learning rate` ضربدر مشتق `loss function` نسبت به آن را از آن کم می‌کند. (بخش اصلی `gradient descent`) که فرق بین `hw4_norm1` و `hw4_sgd` هم در مشتقات اینجاست.

`Predict`: یک نمونه را می‌گیرد، بردار وزن‌ها را در آن ضرب می‌کند و `y` آن را پیش‌بینی می‌کند و بر می‌گرداند.

`Coef_sgd_coefficient`:

در این تابع ابتدا یک مقدار اولیه به وزن‌ها داده می‌شود که تصادفی است. بعد به تعداد تکرارهایی که با دیتای ولیدیشن این تعداد را بهینه کرده‌ایم، `SGD` اجرا می‌شود. به این صورت که اول ۱۰ درصد از داده‌ها به تصادف انتخاب می‌شوند، بعد `y` آنها پیش‌بینی می‌شود و بعد بر اساس اختلاف `y` پیش‌بینی شده با واقعی مقدار `w` آپدیت می‌شود. در هر دور اجرا میزان خطای داده‌های ولیدیشن و تست به ازای بردار وزن‌های آن دور محاسبه می‌شود و در نهایت نمودار تغییرات خطای پیش‌بینی داده‌های ولیدیشن و تست در طول زمان نمایش داده می‌شود.

در آخر هم این تابع فراخوانی می‌شود و `y`های پیش‌بینی شده و واقعی داده‌ی تست در یک `scatterplot` نمایش داده می‌شوند.

پارامترهای `learning rate` و تعداد نمونه در هر دور (`batch size`) چند بار تغییر داده شدند و پارامترهایی که به ازای آنها خطای کمتری روی داده‌ی ولیدیشن بدست آمد انتخاب شدند.

فایل hw4_ridge:

تا خط ۵۰ که با بقیه مشترک است، بعد از آن یک کلاس RidgeRegressor تعریف می‌کنیم که کار رگرسیون را برای ما انجام دهد (برای سهولت جلوی تعریف پارامترها معنی آن کامنت شده)

تابع fit: در این تابع (با GD هم می‌توان مقدار وزن‌ها را به دست آورد که در کامنت شده است. ابتدا یک مقدار اولیه به وزن‌ها داده می‌شود که صفر است. بعد به تعداد تکرارهایی که با دیتای ولیدیشن این تعداد را بهینه کرده‌ایم، GD اجرا می‌شود. در نهایت نمودار تغییرات خطای پیش‌بینی داده‌های ولیدیشن و تست در طول زمان نمایش داده می‌شود.)

تابع update_errs: در هر دور اجرا میزان خطای داده‌های ولیدیشن و تست به ازای بردار وزن‌های آن دور محاسبه می‌شود.

تابع update weights: بعد بر اساس اختلاف y پیش‌بینی شده با واقعی، مقدار w با کم کردن learning rate ضربدر مشتق loss function بر حسب w از آن آپدیت می‌شود.

تابع predict: داده‌ها را می‌گیرد (x و y)، بعد با ضرب کردن بردار وزن‌ها در x آنها، y آنها پیش‌بینی می‌شود.

تابع ridge test: مقدار پارامتر lambda که ضریب پشت جمع 2 norm ها در loss function است را می‌گیرد، یک نمونه از کلاس ridgeRegressor می‌سازد و آن را با داده‌ی train، آموزش می‌دهد و نتایج آن را نشان می‌دهد. سپس مشابه دو فایل دیگر، yهای پیش‌بینی شده و واقعی داده‌ی تست در یک scatterplot نمایش داده می‌شوند.

پارامتر learning rate چند بار تغییر داده شد و پارامتری که به ازای آن خطای کمتری روی داده‌ی ولیدیشن بدست آمد انتخاب شد.

نتایج و خواسته‌های سوالات:

در هر بخش هنگام اجرای برنامه، MSE نهایی مربوط به داده‌های ولیدیشن و تست در خروجی چاپ می‌شود.

مقایسه‌ی وزن‌های بخش ۱ و ۳ (برای این کار فایل combine.py را استفاده کردم تا بتوانم هر دو مدل را با هم run کنم. در این فایل تابع‌ها شماره بخش که ۱ یا ۳ است را ورودی می‌گیرند تا تابع مربوط به آن بخش را اجرا کنند. این هم به جز در هنگام محاسبه‌ی مشتق loss function فرقی با دو فایل hw4_sgd و hw4_norm1 ندارد) نشان می‌دهد که در اکثر موارد، اندازه‌ی وزن‌ها هنگامی که از norm2 استفاده می‌شود بیشتر از زمانی است که از norm1 استفاده می‌شود. یک مشاهده‌ی دیگر این بود که بیشتر وقت‌ها خطای داده‌ی تست در حالت استفاده از norm2 از خطای داده‌ی تست هنگام استفاده از norm1 کمتر می‌شود. (البته به batchای تصادفی‌ای که sgd برای train کردن مدل می‌سازد بستگی دارد که چقدر به داده تست شبیه باشند)

برای رسیدن به این نتیجه در یک حلقه ۱۰ بار هر دو مدل را train و تست کردم.

مقایسه‌ی ridge regression هنگامی که لامبدا برابر ۱ است با هنگامی که لامبدا برابر صفر است (یا همان least squares معمولی) نشان می‌دهد که استفاده از 2 norm در loss function خطای پیش‌بینی در داده‌ی تست را کم می‌کند. Ridge regression این مزایا را دارد:

1. اولاً با کوچک‌تر کردن اندازه‌ی وزن‌ها (parameter shrinkage) به حل مشکل multicollinearity کمک می‌کند
2. با کوچک تر کردن ضرایب، جلوی overfitting را می‌گیرد
3. با کم کردن واریانس، خطای مدل در پیش‌بینی y داده‌ی تست را کم می‌کند.